

ADVANCED RAG SERIES

Reranking from Scratch

BM25 Lexical + Cross-Encoder Neural + Cohere API + 2026 Frontier

by Mayank Chugh

@itaienthusiast | IT AI Enthusiast

What Is Reranking – and Why Does It Matter?

Reranking is a second-stage filter applied to the top-K documents retrieved by a bi-encoder. A more powerful model re-scores each query-document pair to surface the truly relevant results before they reach the LLM.

Without Reranking

- Top-K docs may include noise
- Bi-encoder misses exact keyword matches
- All top-K chunks sent to LLM -- too many tokens
- LLM gets confused by low-relevance context
- Hallucinations increase with noisy input
- No relevance score -- hard to filter

With Reranking

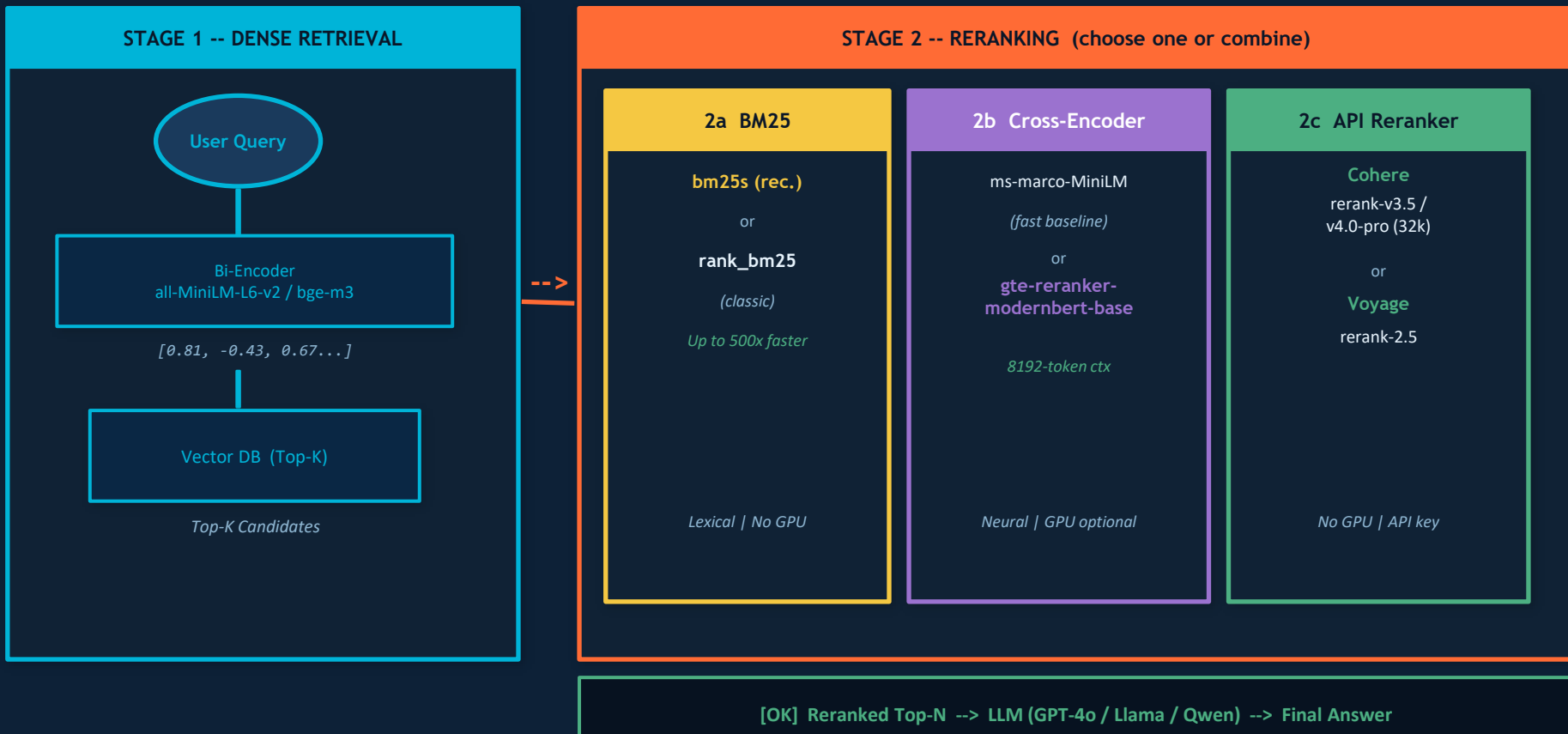
- Two-stage filter removes noise
- Cross-encoder reads query + doc together
- Only top reranked docs reach the LLM
- Tight, high-signal context = better answers
- Measurable relevance score (0.0-1.0)
- Works with BM25 + dense or either alone

Company interview analogy: aptitude test (retrieval) -> group discussion (reranking) -> final interview (LLM)

Teaching note: The college interview analogy from the original video: 100 candidates (corpus) -> aptitude test (retrieval top-K) -> group discussion (reranking) -> final interview (LLM). Each stage filters further.

Two-Stage RAG Reranking Pipeline

Advanced RAG Series 2026 | by Mayank Chugh | @itaienthusiast



Teaching Note: Two-stage = speed + accuracy. Stage 1 filters millions of docs fast; Stage 2 ranks the small filtered set with full context.

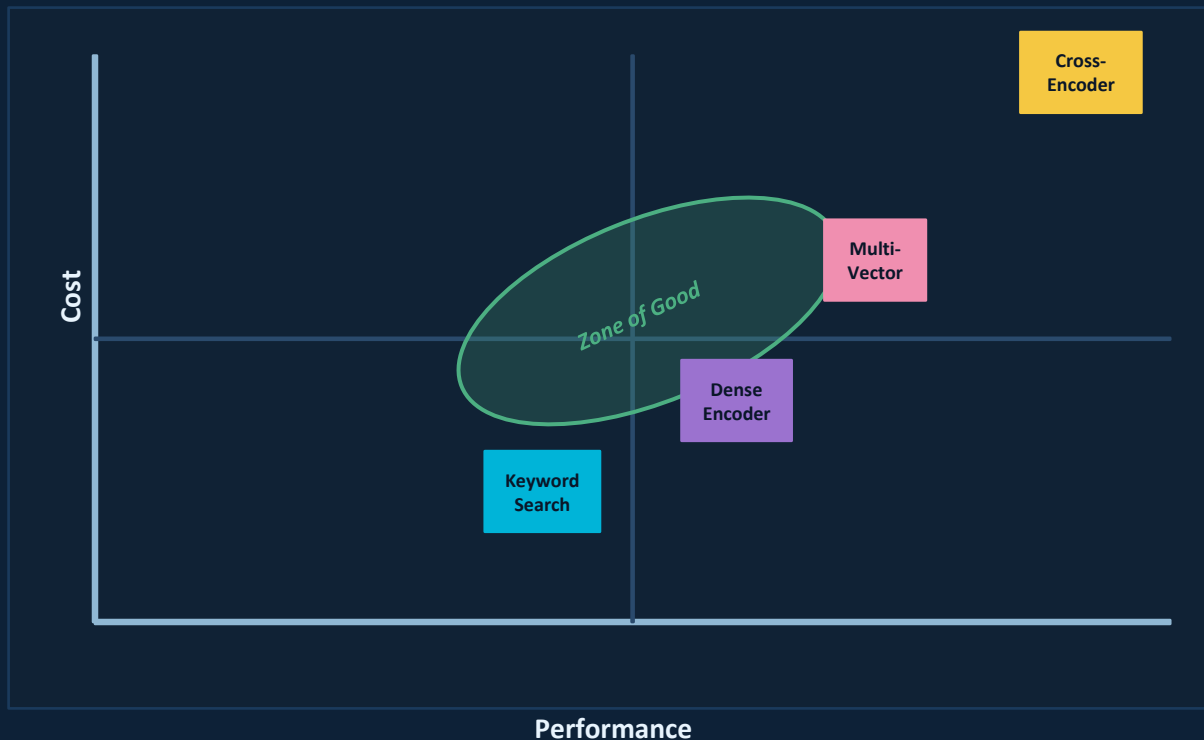
Reranking Methods -- The Full Landscape

Model	Type	Performance	Cost	Example
Cross-Encoder	Open source	Great	Medium	BGE, sentence-transformers, Mixedbread
Multi-Vector	Open source	Good	Low	ColBERT
LLM	Open source	Great	High	RankZephyr, RankT5
LLM API	Private	Best	Very High	GPT, Claude
Rerank API	Private	Great	Medium	Cohere, Mixedbread, Jina

Today's episode covers: Cross-Encoder (row 1) + Rerank API / Cohere (row 5) | Plus BM25 (keyword search baseline, not shown here)

Teaching note: Multi-Vector (ColBERT), LLM-based (RankZephyr, RankT5), and LLM API rerankers are covered in later episodes -- Lost in the Middle, Rank Fusion, and FlashRank.

Retrieval -- Performance vs Cost Trade-off



Reading the Chart

Bottom-left:
low cost, low
performance

Top-right:
high cost, high
performance

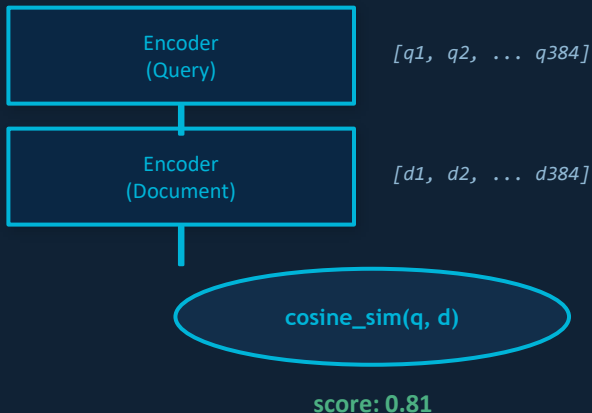
Zone of Good:
best perf-to-
cost ratio

Teaching note: No free lunch: performance generally trades off against cost. Multi-vector (ColBERT) and dense encoders sit in the 'zone of good' -- strong performance without cross-encoder-level cost.

Bi-Encoder vs Cross-Encoder -- Architecture Contrast

by Mayank Chugh | @itaienthusiast

BI-ENCODER (Stage 1 -- Retrieve)



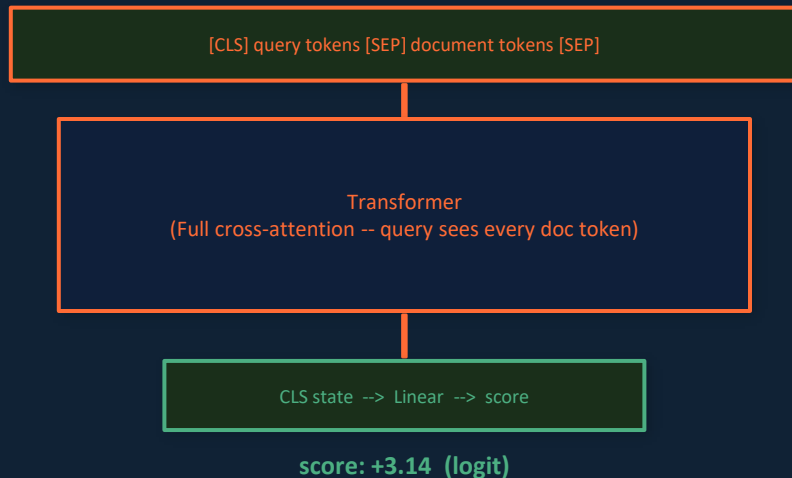
[OK] Fast -- pre-compute doc embeddings

[OK] Scales to millions of docs

[!] Less accurate -- encoded separately

Model: all-MiniLM-L6-v2 / bge-m3

CROSS-ENCODER (Stage 2 -- Rerank)



[OK] Accurate -- full joint attention

[OK] Best for reranking top-k

[!] Cannot pre-compute -- 1 pass per pair

Model: ms-marco-MiniLM / gte-modernbert

Pipeline Architecture

Phase 1 Dense Retrieval

8 Sample
Documents

all-MiniLM-L6-v2
384-dim embed

Cosine Similarity
query vs all docs

Ranked Top-4
Candidates

User Query

Phase 2a BM25 Lexical

Tokenize
doc.split()

BM25Okapi
(rank_bm25)

bm25s
(500x faster)

BM25 Scores
unbounded float

Phase 2b Cross-Encoder

Sentence Pairs
[query, doc]

CrossEncoder
ms-marco-MiniLM

.predict() or
.rank() (v5 new)

Raw Logit Scores
can be negative

Phase 2c Cohere API

cohere.ClientV2()
(2026 required)

rerank-v3.5
or v4.0-pro (32k)

return_documents
=True (required)

Scores 0-1
normalised

Phase 3 Consensus

BM25
Rank-1

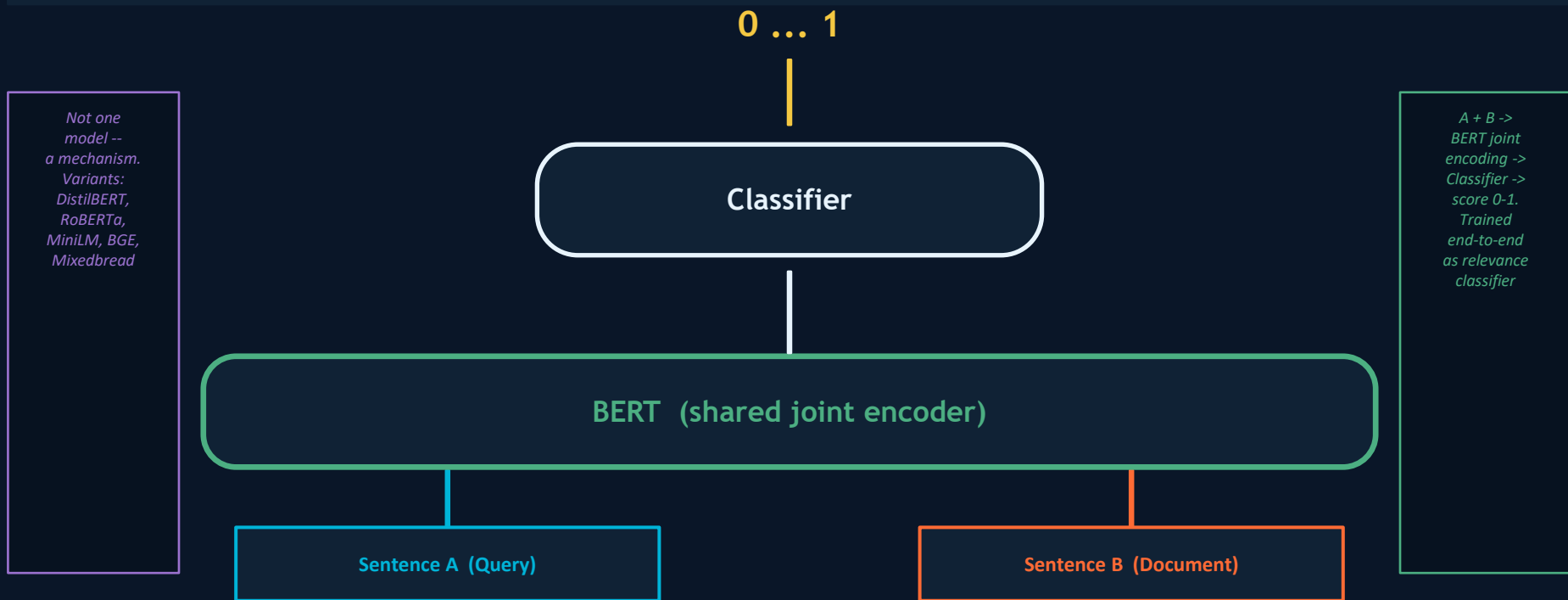
CrossEnc
Rank-1

Cohere
Rank-1

Top-2 to
LLM

Inside the Cross-Encoder

The input of the model always consists of a data pair -- for example two sentences -- and outputs a value between **0 and 1** indicating the similarity between these two sentences.



Teaching note: Unlike a bi-encoder's cosine similarity (a generic vector-space metric), the Cross-Encoder classifier is trained end-to-end specifically to predict relevance -- which is why it consistently outperforms bi-encoders on reranking tasks.

2026 Reranker Landscape -- What Is New

Like & Subscribe!

Model	Params	BEIR nDCG@10	License	Cost / Usage
jina-reranker-v3	0.6B	61.94 -- SOTA	CC-BY-NC (!)	API (\$) or self-host GPU
Qwen3-Reranker-8B	8B	Top open family	Apache 2.0 [v]	Self-host / vLLM
mxbai-rerank-large-v2	1.5B	61.44	Apache 2.0 [v]	pip install mxbai_rerank
BAAI/bge-reranker-v2-m3	568M	Strong multi.	Apache 2.0 [v]	pip install FlagEmbedding
Voyage rerank-2.5	--	+7.9% vs Cohv3.5	Proprietary	\$0.05/1M tokens, 200M free
Cohere rerank-v4.0-pro	--	ELO ~1627	Proprietary	Model Vault / per-search
Cohere rerank-v3.5	--	ELO ~1457	Proprietary	\$2/1k searches
ms-marco-MiniLM [old]	22M	Dated baseline	Apache 2.0 [v]	Free, pip install ST

(!) = CC-BY-NC: non-commercial only without paid agreement | [v] = Apache 2.0: safe for commercial products | [old] = year-old baseline

Teaching Note: For commercial open-source always check Apache 2.0. For max quality use jina-v3 or Qwen3-8B. For no-GPU use Cohere or Voyage APIs.

Score Types -- Do Not Compare Raw Numbers Across Methods

BM25 Scores

`rank_bm25 / bm25s`

Rank 1: 1.4580

Rank 2: 1.2751

Rank 3: 0.4369

Rank 4: 0.0000

- Unbounded positive float
- Higher = more keyword overlap
- Keyword-level tokens only
- Cannot be negative
- No semantic understanding

Cross-Encoder Scores

`CrossEncoder.predict()`

Rank 1: +3.1379

Rank 2: +0.8422

Rank 3: -2.8782

Rank 4: -2.9193

- Raw logit -- can be NEGATIVE
- Higher = more relevant
- Not a probability (no sigmoid)
- Bar chart: normalise before display
- v5: `.rank()` returns sorted directly

Cohere Scores

`cohere.ClientV2.rerank()`

Rank 1: 0.9621

Rank 2: 0.7834

Rank 3: 0.4152

Rank 4: 0.1087

- Normalised 0-1 range
- Query-dependent scale
- Do NOT compare across queries
- `return_documents=True` required
- v3.5: 4096 ctx | v4.0-pro: 32k ctx

KEY RULE: Only compare RANKINGS across methods -- not raw score values. Each method uses a completely different scale.

Teaching note: The scores shown are actual outputs from running the notebook on the 8-document corpus with query: 'Natural language processing techniques enhance keyword extraction efficiency.'

How to Choose Your Reranker -- 2026 Decision Guide

Commercial Product?

Yes -->

<-- No

Need multilingual?

Yes -->

<- No

Have GPU?

Yes -->

<- No

OSS + multilingual

mxbai-rerank-v2
bge-reranker-v2-m3
Qwen3-Reranker

No GPU (API)

Cohere
rerank-v3.5
Voyage
rerank-2.5

OSS Apache 2.0

mxbai-rerank-v2
bge-reranker-v2-m3
Qwen3-Reranker

SOTA open (CC-NC)

jina-reranker-v3
61.94 BEIR SOTA
Cohere v3.5

Need multilingual?

Yes -->

<- No

Have GPU?

Yes -->

<- No

OSS + multilingual

mxbai-rerank-v2
bge-reranker-v2-m3

API (no GPU)

Cohere v3.5
Voyage
rerank-2.5

Long docs (>4k tokens)? --> Cohere v4.0-pro / v4.0-fast (32k ctx) | jina-reranker-v3 (131k ctx)
Latency-critical production? --> Cohere v4.0-fast | Voyage rerank-2.5-lite | Ettin 17M (CPU sub-ms)

Key Takeaways + What's Next

Like & Share & Subscribe!

[OK] Reranking methods landscape: Cross-Encoder, Multi-Vector (ColBERT), LLM (RankZephyr/T5), LLM API, Rerank API -- pick based on the performance-vs-cost trade-off

[OK] Update Cohere: `cohere.Client()` -> `cohere.ClientV2()` | `rerank-english-v3.0` -> `rerank-v3.5` | add `return_documents=True`

[OK] Use bm25s for production BM25 (up to 500x faster than `rank_bm25` on large corpora -- BM25S paper arXiv:2407.03618)

[OK] 2026 frontier: jina-reranker-v3 (SOTA BEIR 61.94) | Qwen3-Reranker-8B (Apache 2.0) | mxbai-rerank-v2 (RL-trained)

[OK] LangChain v1.0 imports changed: from `langchain_cohere` import `CohereRerank` | from `langchain_classic` import ...

[>>] Coming Next: Multi-Vector (ColBERT) + LLM-based Reranking (RankZephyr) + Lost in the Middle -- Episode 05 | GitHub: github.com/mayankchugh-learning